Computer Graphics Using OpenGL
INM376 / IN3005

Module Leader – Dr Eddie Edwards

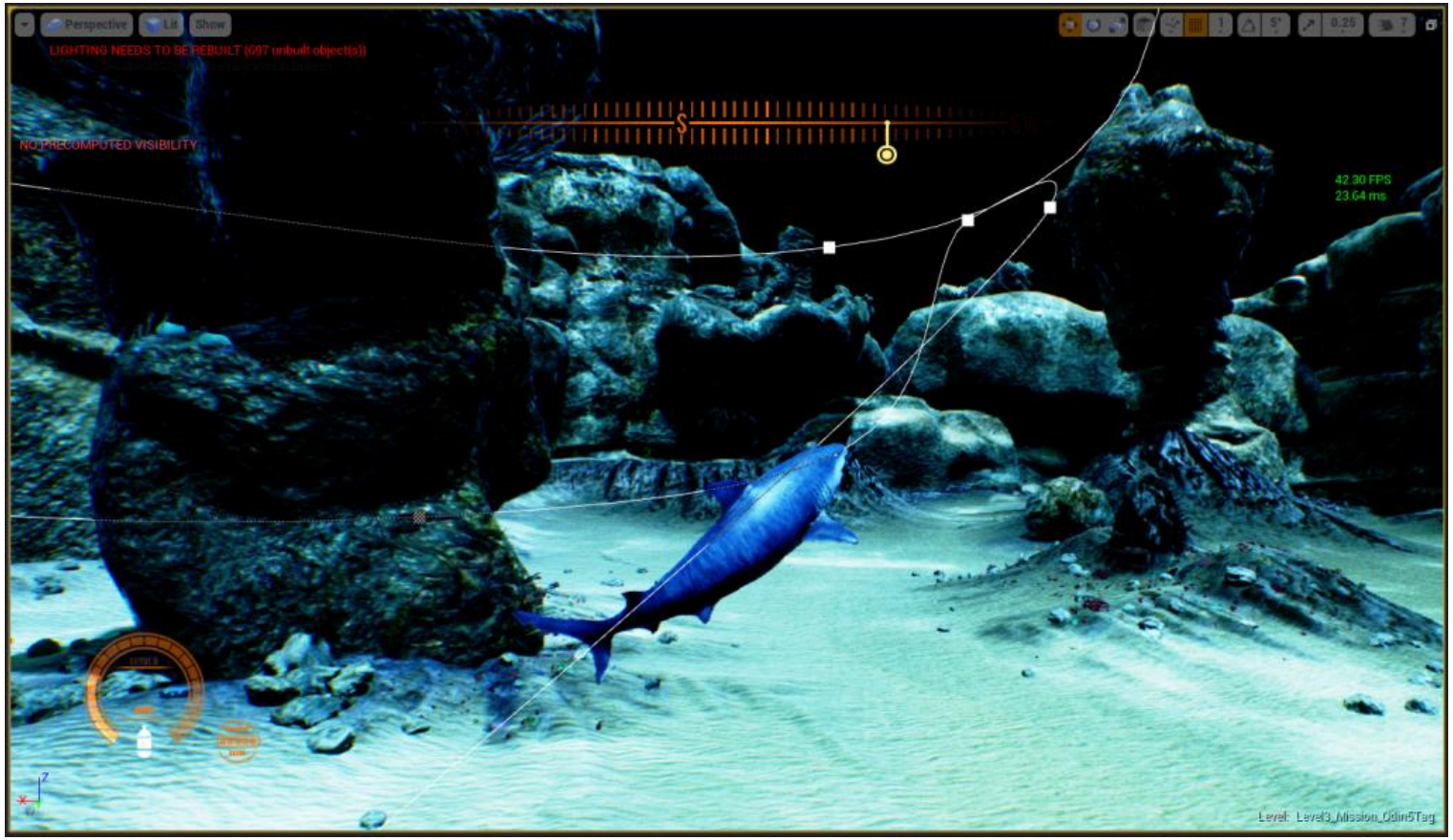
Lecture 08: Intermediate / Advanced Graphics 1

Recap from last time

- Buffers
  - Colour
  - Depth
  - Stencil
  - Frame buffer objects (FBOs)
- Image processing (screen space techniques)
  - Various types of blur
  - Bloom
  - Vignetting
- Perlin noise

Life in “the real world”

- Former student working at Beefjack on a game called *Iron Fish*
- Programming AI for aquatic NPCs, using Catmull-Rom splines to set the paths

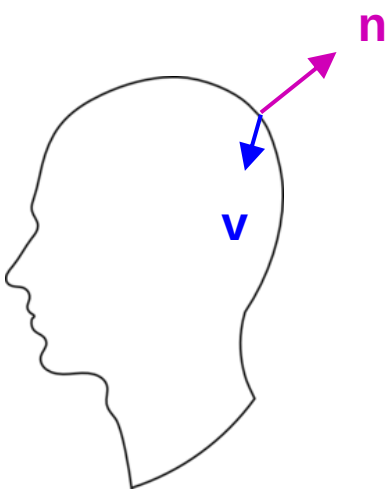


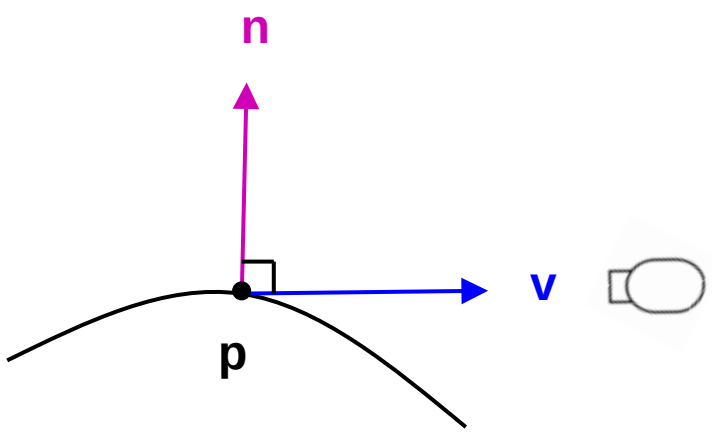
Overview of today's lecture

- Noise
- Advanced texture mapping
  - Environment mapping using cubemap textures
  - Bump mapping
- Fog
- Shadows
- Instanced rendering
- Coursework submission

A better toon shader

- In lecture and lab 4 we implemented a toon shader, which used a few levels of colour to produce a cartoon-like appearance.
- We can improve this toon shader by including a black line around the object by finding its *silhouette*. Recall a silhouette is the outline of the object as observed from a viewer.
- Questions:
  - Does the silhouette change if the person moved?
  - Does the silhouette change if the camera moved?
  - Does the silhouette change if a light source moved?

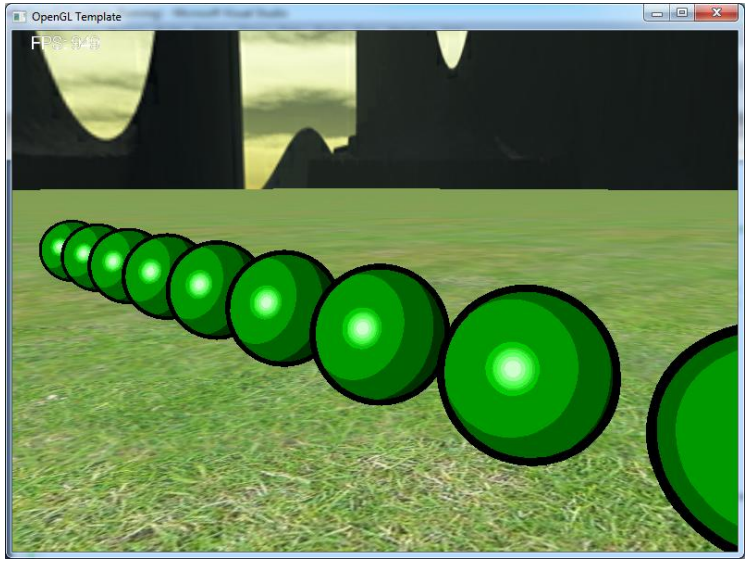




A better toon shader

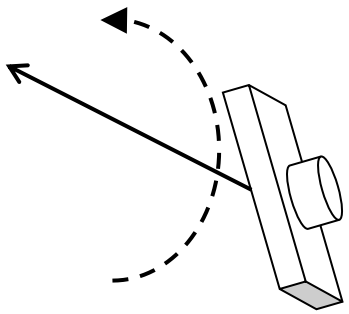
- All we need to do is look at the dot product of the *normal* and *viewing* vectors.
- If it is below some threshold, set the fragment to black!

```
vec3 n = normalize(n);
vec3 v = normalize(-p.xyz);
float edgeMask = (dot(v, n) <  0.4) ? 0 : 1;
vec3 toonColour = edgeMask * quantisedColour;
```



Camera shake

- Camera shake perturbs the camera
- Shake can result in a translation or rotation of camera
- However, the skybox is designed to appear be stationary with respect to camera translations. So if you implement a camera shake, it may be more effective to perturb *rotation* than *translation*
- There are many ways to implement this. One suggestion would be to compute small, random rotation angles, and linearly interpolate over time to animate.



Environment mapping

- *Environment mapping* is a texture mapping technique used to simulate reflection or refraction by modelling the background environment.
- The texture stores the appearance of the distant environment surrounding the object. This is typically implemented as a *cubemap texture*, which consists of six images arranged on the sides of a cube (like a skybox)
  - In *reflection mapping*, texture coordinates simulate the reflection of the light off the surface. Example: Shiny guy in Terminator 2.
  - In *refraction mapping*, texture coordinates simulate the refraction of the light through the surface.

Cubemap textures

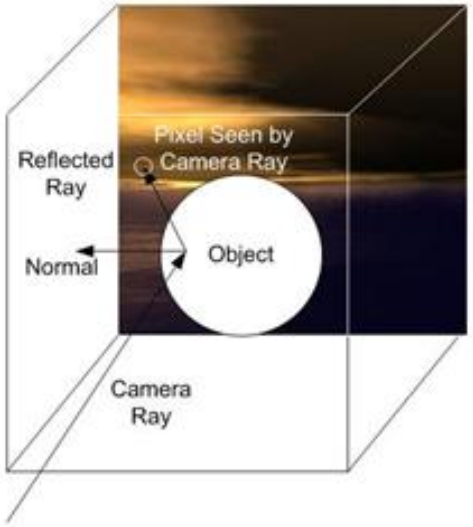
- A cube map is a single texture object that consists of six square 2D images.
- The images make up the sides of a cube.
- Cubemap textures are useful for rendering skyboxes and performing environment mapping.
- When loading a cubemap texture, one loads the six images using glTexImage2D, but instead changing the target from GL_TEXTURE_2D to one of the following:

GL_TEXTURE_CUBE_MAP_POSITIVE_X	GL_TEXTURE_CUBE_MAP_NEGATIVE_X
GL_TEXTURE_CUBE_MAP_POSITIVE_Y	GL_TEXTURE_CUBE_MAP_NEGATIVE_Y
GL_TEXTURE_CUBE_MAP_POSITIVE_Z	GL_TEXTURE_CUBE_MAP_NEGATIVE_Z

- Depending on your images, you may have to flip them: http://stackoverflow.com/questions/11685608/convention-of-faces-in-opengl-cubemapping
- There is a CCubeMap class in the template code that handles this.

Cubemap texture coordinates

- In the fragment shader, cubemap textures are accessed using a *samplerCube*.
- 3D texture coordinates, $[s, t, r]^T$ are used in conjunction with the `samplerCube`.
- Each texture coordinate is interpreted as a direction vector from the centre of the cube. The direction vector is normally in world coordinates.
- The line defined by the vector and the centre of the cube is extended to intersect one of the cube faces.
- The image that corresponds to that face is then accessed at the location of the intersection.



Rendering a skybox with a cubemap

- One can render a skybox using a cubemap texture.
- For texture coordinates, one simply provides the cube vertex positions (in world coordinates), assuming the cube is centred at the origin.
- The vertex positions then define vectors from origin.

vertex shader

```
out vec3 worldPosition;
void main() {
   gl_Position = matrices.projMatrix * matrices.modelViewMatrix *
                 vec4(inPosition, 1.0f);
   worldPosition = inPosition;
}
```

fragment shader

```
in vec3 worldPosition;
uniform samplerCube CubeMapTex;
void main() {
   vOutputColour = texture(CubeMapTex, worldPosition);
}
```

Environment mapping: reflection

- To implement reflection, one must determine how the viewing vector from the camera reflects off the surface, using the `reflect` function in GLSL.
- Since the reflected vector must be in world coordinates, there are two options:
  - Perform the reflection in world coordinates
    - This is the approach taken In the OpenGL Shading Language Cookbook
    - It requires knowledge of the model matrix (independent of the view matrix)
  - Perform the reflection in eye coordinates, then rotate the reflected point to world coordinates.
    - It requires knowledge of the inverse view matrix.

```
glm::mat4 inverseViewMatrix = glm::inverse(pCamera->GetViewMatrix());
pEMProgram->SetUniform("matrices.inverseViewMatrix", inverseViewMatrix);
```

Environment mapping: reflection

vertex shader

```
out vec3 reflected;
void main() {
   gl_Position = matrices.projMatrix * matrices.modelViewMatrix *
         vec4(inPosition, 1.0f);
   // Get the vertex normal and vertex position in eye coordinates
   vec3 n = normalize(matrices.normalMatrix * inNormal);
   vec3 p = (matrices.modelViewMatrix * vec4(inPosition, 1.0f)).xyz;
   reflected = (matrices.inverseViewMatrix * vec4(reflect(p, n), 1)).xyz;
}
```

fragment shader

```
in vec3 reflected;
uniform samplerCube cubeMapTex;
out vec4 vOutputColour;
void main() {
   vOutputColour = texture(cubeMapTex, normalize(reflected));
}
```

Example

Environment mapping: refraction

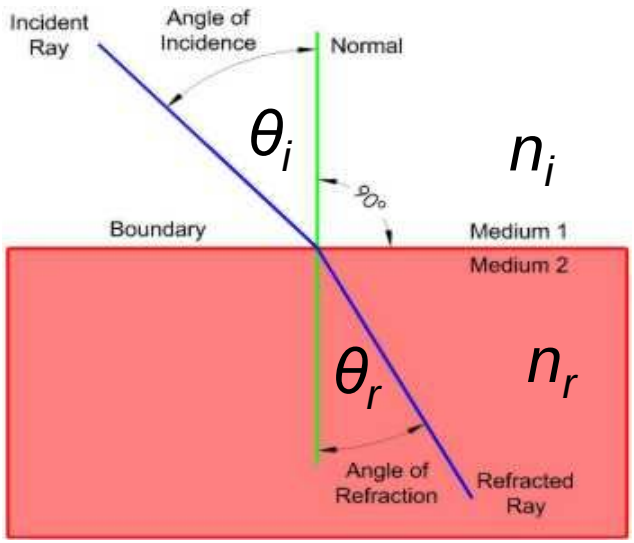
- Refraction is the change in direction of light that occurs when it passes through materials with different refractive indices.

Environment mapping: refraction

- Refraction is well studied in physics, and is described by Snell's law, which relates the angle of incidence θ_i to the angle of refraction, θ_r as

$$\frac{\sin\theta_r}{\sin\theta_i} = \frac{n_i}{n_r} = \eta$$

- Where n_i and n_r are the indices of refraction of the respective media.
- A value of η = 0.66 corresponds to an air / glass interface



http://www.mysundial.ca/tsp/refraction_of_light.html

Environment mapping: refraction

vertex shader

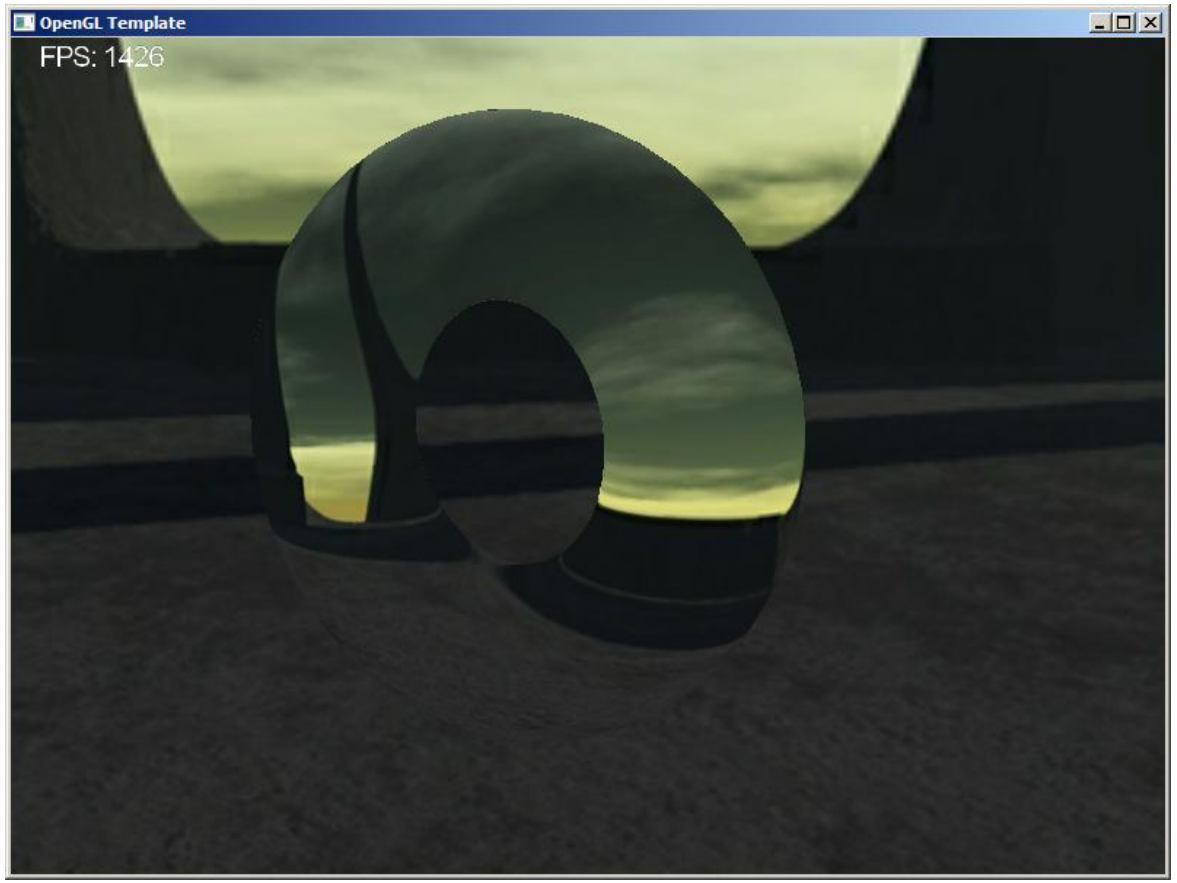
```
out vec3 reflected;
void main() {
  gl_Position = matrices.projMatrix * matrices.modelViewMatrix *
                vec4(inPosition, 1.0f);

  // Get the vertex normal and vertex position in eye coordinates
  vec3 n = normalize(matrices.normalMatrix * inNormal);
  vec3 p = (matrices.modelViewMatrix * vec4(inPosition, 1.0f)).xyz;
  reflected = (matrices.inverseViewMatrix * vec4(refract(p, n, eta), 1)).xyz;
}
```

fragment shader

```
in vec3 reflected;
uniform samplerCube cubeMapTex;
out vec4 vOutputColour;
void main() {
   vOutputColour = texture(cubeMapTex, normalize(reflected));
}
```

Example

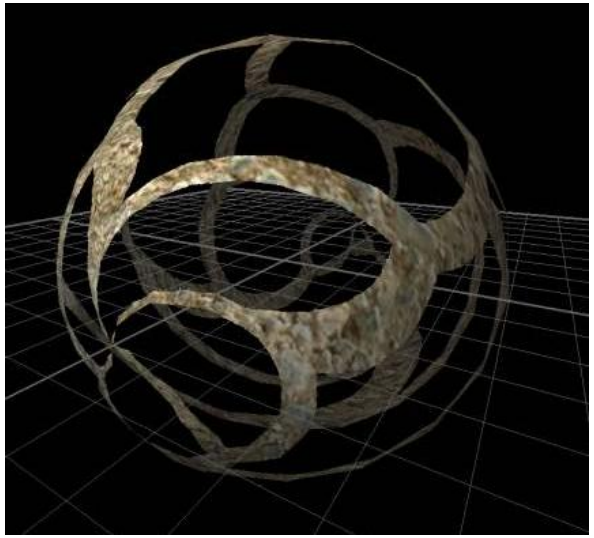


⇨ See the Cookbook, chapter 4 for more details on environment mapping

Alpha maps

- During multi-texturing, one can load an RGBA texture, and discard any fragments whose alpha value is below a threshold

alpha map



texture mapped surface

http://www.ozone3d.net/tutorials/glsl_texturing_p05.php

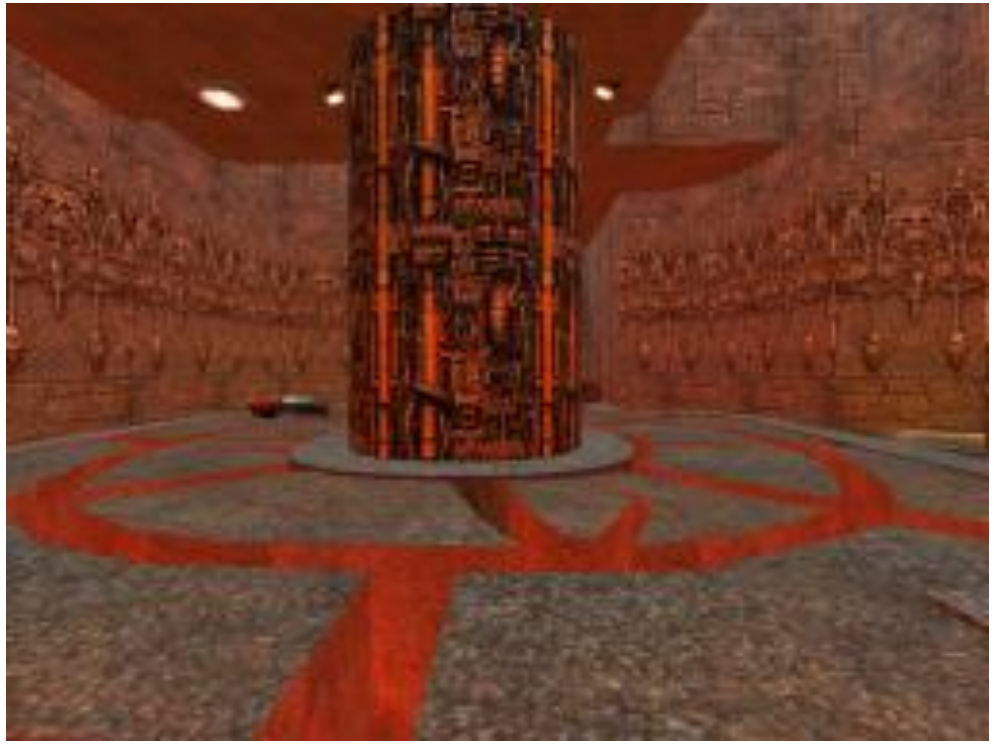
Lightmapping

- In lightmapping, the amount of light reaching an object is stored as a texture (the lightmap).
- The lightmap is typically pre-computed and applied to static objects. The process of pre-computing the lightmap is sometimes referred to as *baking*.
- During rendering, the lightmap is applied through multi-texturing to produce shading.

lightmap texture lit texture

- Quake was the first computer game to use lightmapping.
- Lightmapping provides an efficient way to simulate the effect of more lights.

Example from Quake 3

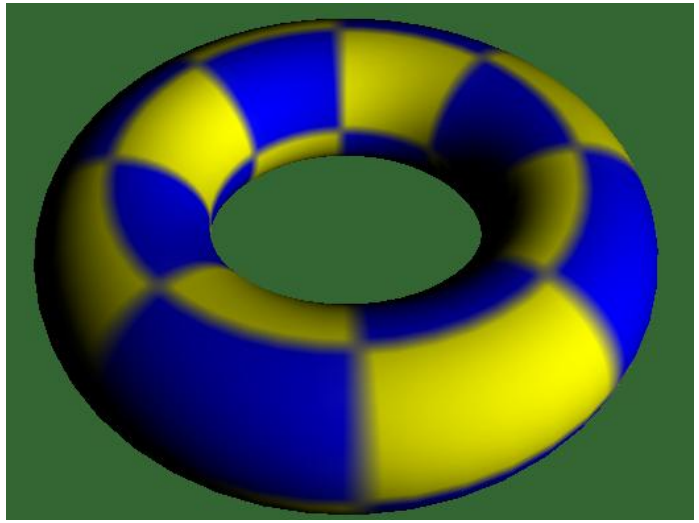


lightmapping off

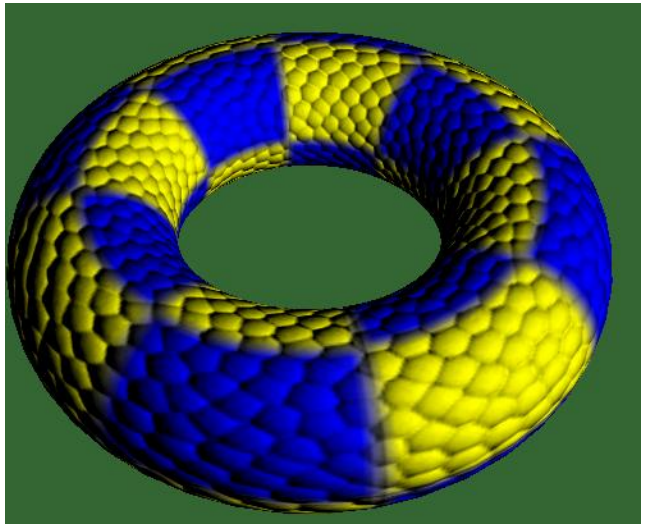
lightmapping on

Bump mapping

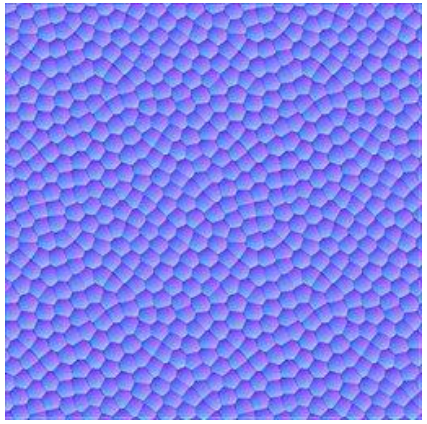
- Bump mapping (aka normal mapping) is technique for simulating bumps and wrinkles on the surface of an object.
- It is achieved by perturbing the surface normal for each fragment being rendered. The new normal is used in per-fragment lighting calculations.
- The result is a surface that appears bumpy, even though the underlying position of points on the surface is unchanged.



bump mapping off



bump mapping on



bump map

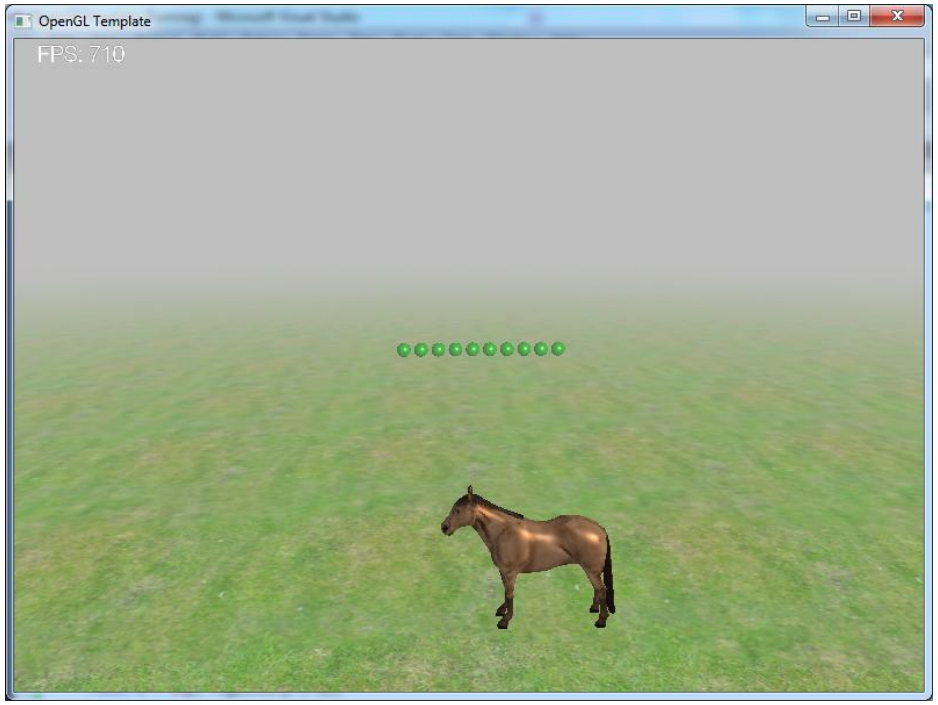
http://www.paulsprojects.net/tutorials/simplebump/simplebump.html

Bump mapping

- Bump mapping can be implemented using multi-texturing. A texture representing the normal displacement can be loaded.
- Displacements are represented as RGB in the texture, in the tangent space of the surface.
- When rendering, the surface normals are displaced based on the texture. The displaced normal is used for calculating lighting.
- See the Cookbook for a recipe

Fog

- Fog is an easy effect to add to a scene.
- It adds realism, and is used to prevent geometry outside the view frustum from suddenly popping into view as the camera moves closer.
- Fog is computed in the fragment shader.



Fog

- Fog blends a colour (usually grey or whitish) with the true colour rendered, i.e. **C** = w **S** + (1 – w) **F**, where **C** is the final colour, **S** is the scene colour without fog, **F** is the fog colour, and w is a blending factor.
- Common fog models include
  - Linear
  - Exponential
  - Square exponential

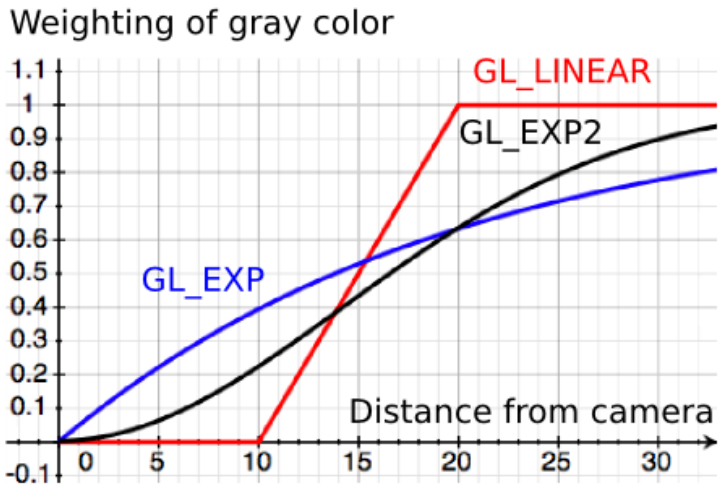
linear

exponential

square exponential

Blending equations

- Linear fog:
  $w = (f_e - d) / (f_e - f_s)$
- Exponential fog:
  $w = \exp(-\rho * d)$
- Square exponential fog:
  $w = \exp(-(\rho * d)^2)$



http://www.videotutorialsrock.com/opengl_tutorial/fog/text.php

where w is the fraction of the original colour to blend
(1-w) is the fraction of gray colour to blend, shown in plot above
f_s is the start fog distance (before this, no fog)
f_e is the end fog distance (beyond this everything is grey)
ρ is the fog density
d is the distance of the point to the camera (z in eye coordinates)

Programming fog in OpenGL

- Exercise: complete the exponential fog shader.
  - Hint 1: w = exp(-ρ * d)
  - Hint 2: mix does linear interpolation: `mix(x, y, w) = x·(1 – w) + y·w`

```
uniform sampler2D sampler0;  // texture sampler
in vec2 vTexCoord; // interpolated texture coordinate
in vec3 n; // normal
in vec4 p; // position in eye coordinates (remember, camera at origin)
float rho = 0.01f;
vec3 fogColour = vec3(0.75f);
...
void main()
{
   vec3 vColour = PhongModel(p, normalize(n));
   vec4 vTexColour = texture(sampler0, vTexCoord);
   vOutputColour = vTexColour*vec4(vColour, 1.0f);

   // Use mix to linearly interpolate fog
   float d = length(p.xyz);
   float w = exp(-rho*d);
   vOutputColour.rgb = mix(fogColour, vOutputColour.rgb, w);
}
```

fragment shader

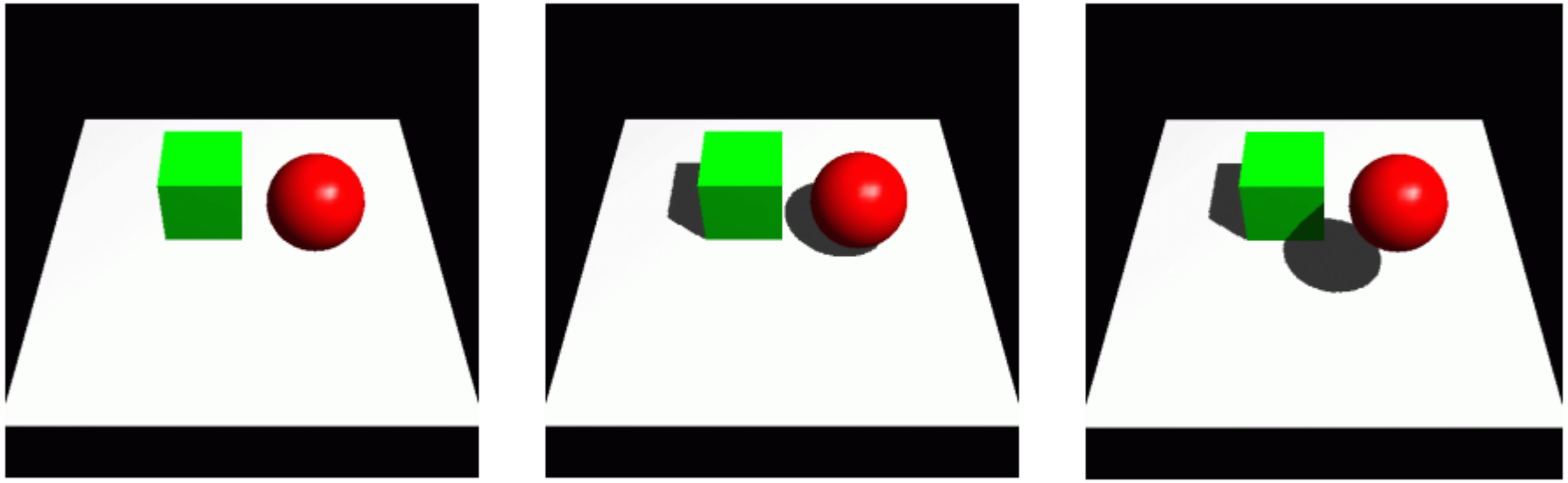
NSS Survey

- Please fill this in
- It is our chance to get feedback on what we can do better
- It’s your chance to tell us what is good and bad about the course
- We will act on your recommendations to improve things next year

https://www.thestudentsurvey.com/

Shadows

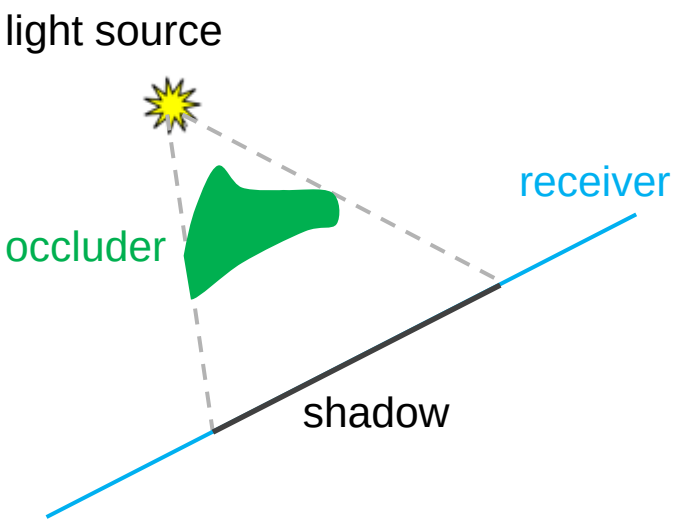
- Shadows enhance realism and help illustrate the spatial relationships between objects in a scene.



http://cg.informatik.uni-freiburg.de/course_notes/graphics_07_shadows.pdf

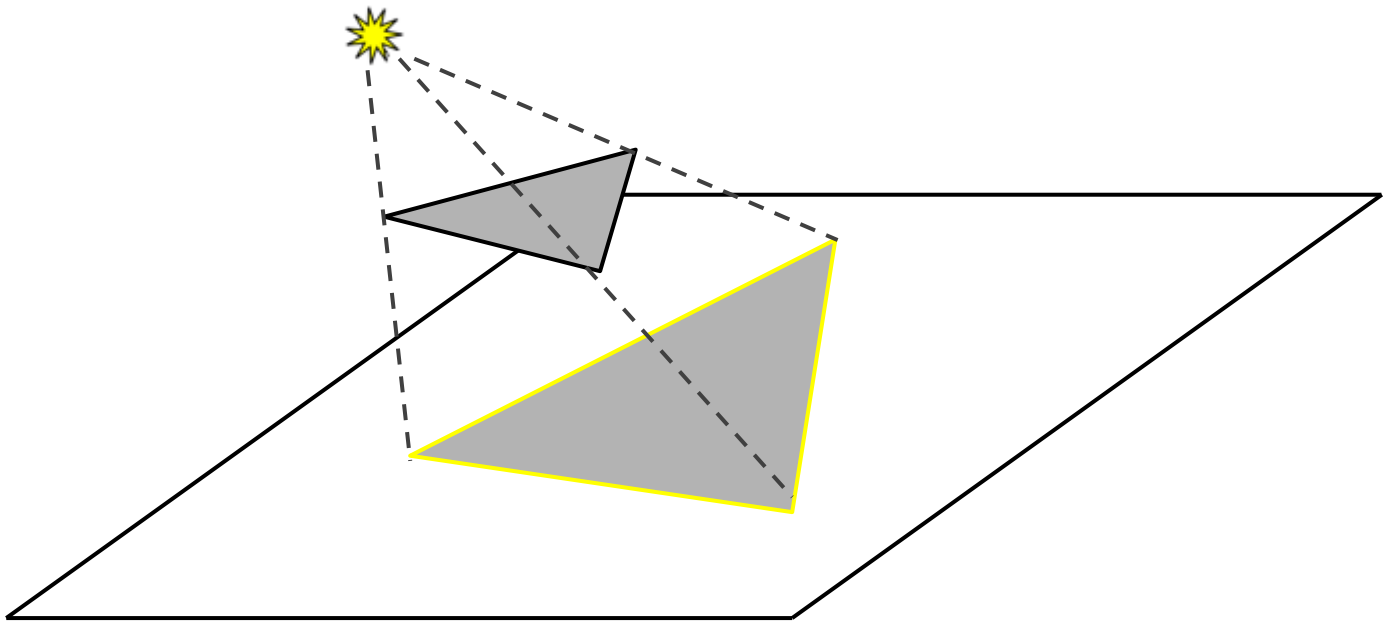
Shadows

- Recall that OpenGL implements a local lighting model. Each primitive is rendered independently of any other primitive.
- Shadows are not part of the standard pipeline in OpenGL because they are *not local.*
- There are numerous ways to implement shadows in computer games, such as
  1. Planar shadows
  2. Shadow mapping
  3. Shadow volumes (sometimes called stencil shadows)
- Terminology



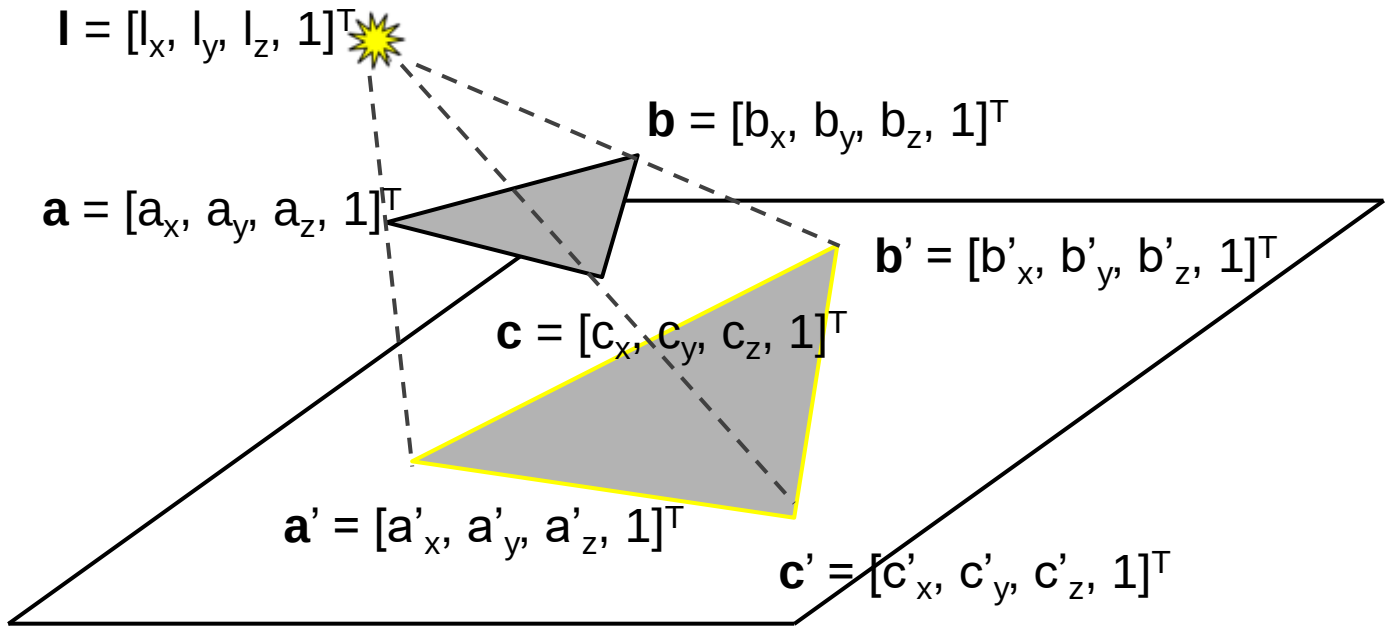
Planar shadows

- Project the occluder primitives from the light source onto a receiver plane, such as the ground
- The projected primitives then form the shadow geometry



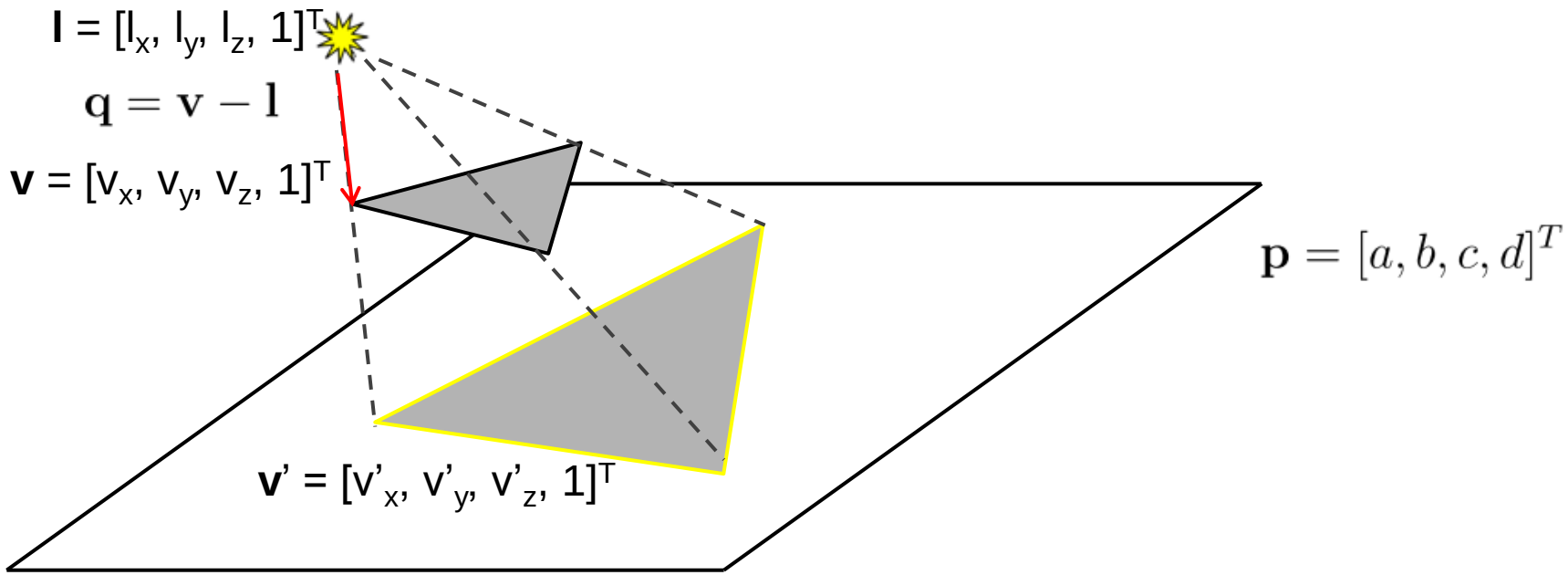
Planar shadows

- Project the occluder primitives from the light source onto a receiver plane, such as the ground
- The projected primitives then form the shadow geometry



- To find the projected vertices, must solve the equation of ray / plane intersection

Planar shadows



- A point along the ray is $\mathbf{r} = \mathbf{l} + \mathbf{q}t$
- The plane equation is given as $ax + by + cz + d = 0$
- Solving for t gives $$t = -\frac{al_x + bl_y + cl_z + d}{aq_x + bq_y + cq_z}$$
- Can plug this back into the equation for **r** to determine the intersecting point **v**'

Planar shadow matrix

- The projected point can be expressed as $\mathbf{v}' = \mathbf{M}\mathbf{v}$

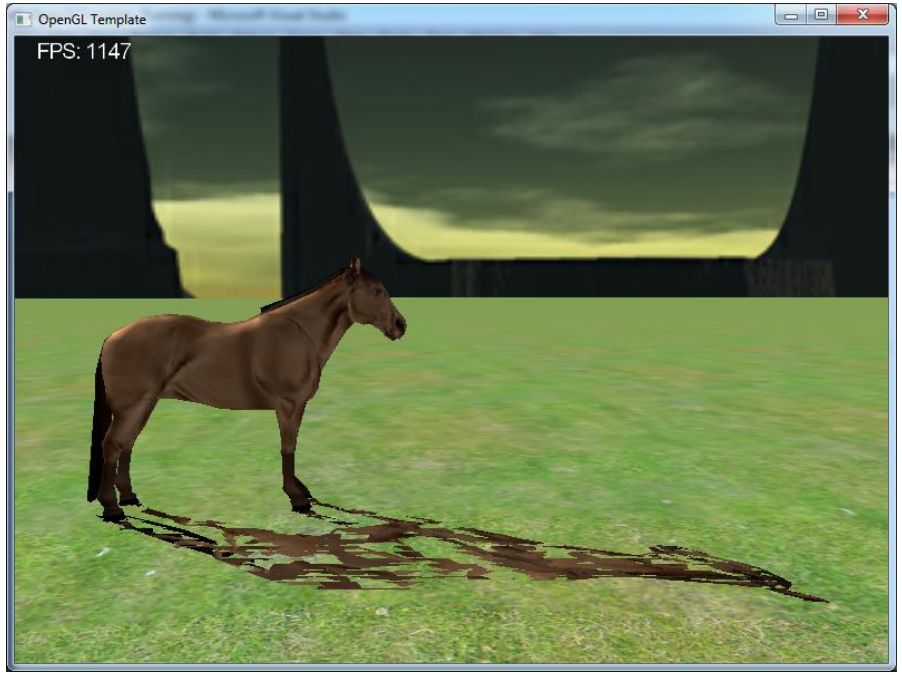
  where $\mathbf{M} = (\mathbf{p} \cdot \mathbf{l})\mathbf{I} - \mathbf{l}\mathbf{p}^T$ *shadow matrix*.

- The projection shadow matrix gives a simple way to project any vertex **v** from a light **l** onto a plane **p**.
- To use, simply form it, and then load it into the matrix stack

```
glm::mat4 planarShadowMatrix = glm::dot(plane, lightPos) * glm::mat4(1) -
        glm::outerProduct(lightPos, plane);
```

Planar shadows

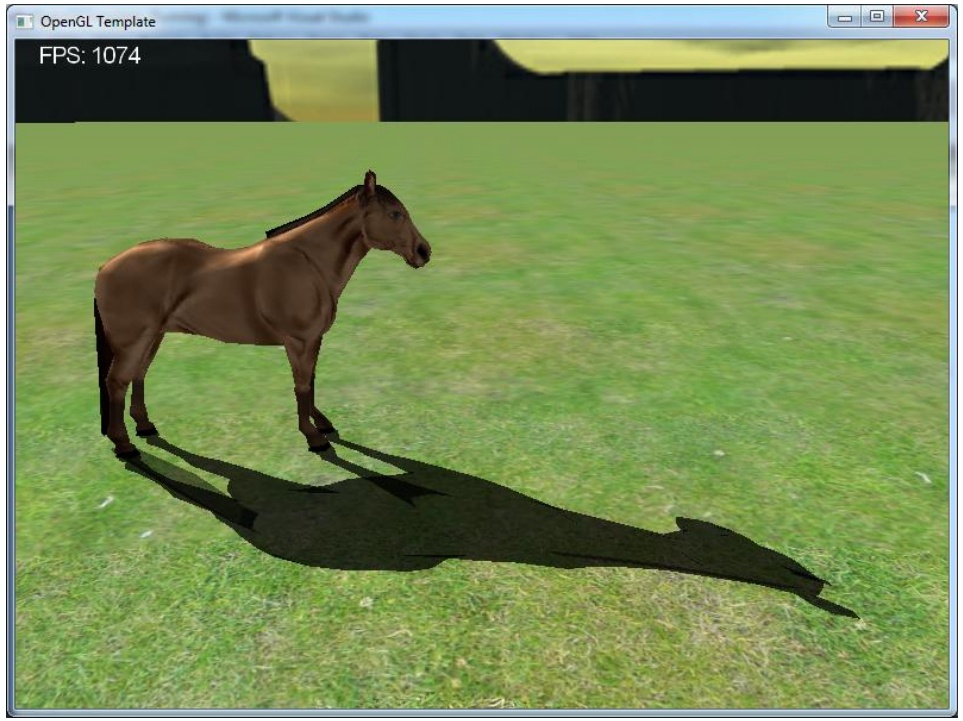
- Projection is right, but rendering is not!



- Issues: z-fighting, shadow is the same colour as the object being rendered

Planar shadows

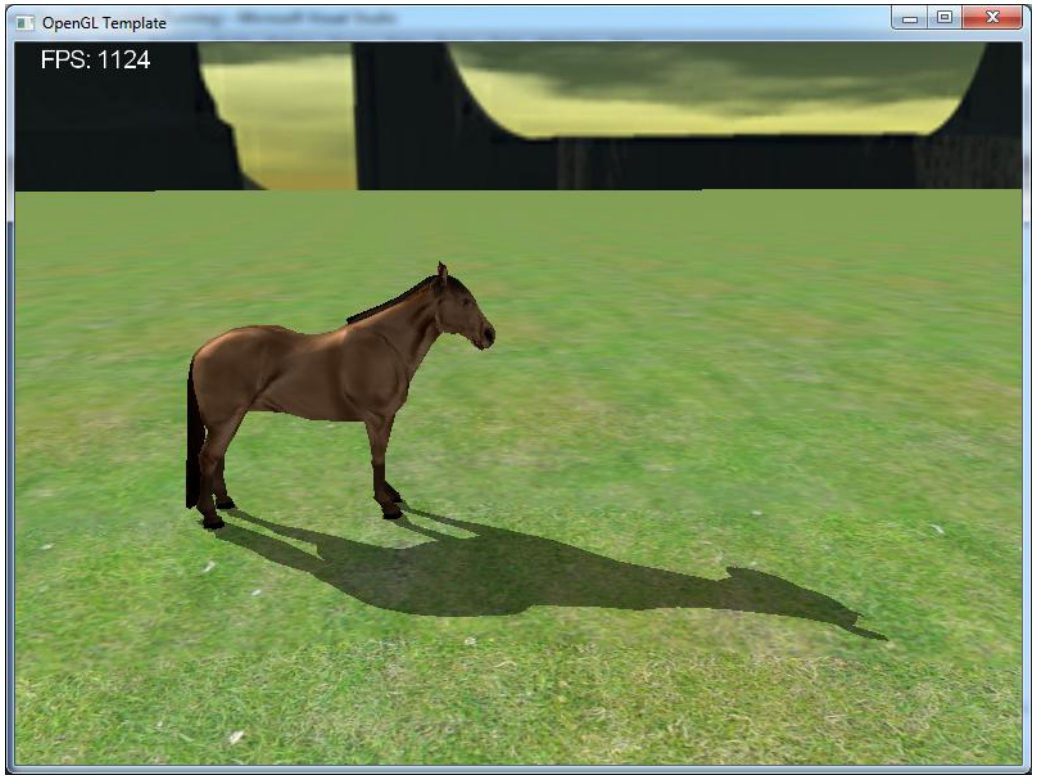
- Solution 1: Alpha blend dark colour, disable depth testing

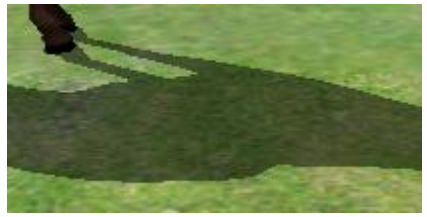


- Pretty good, but multiple shadowing

Planar shadows

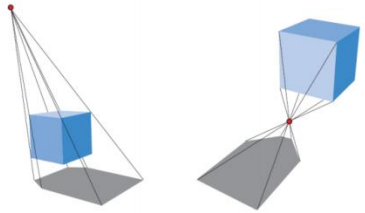
- Solution 2: Use the stencil buffer to only darken a fragment once



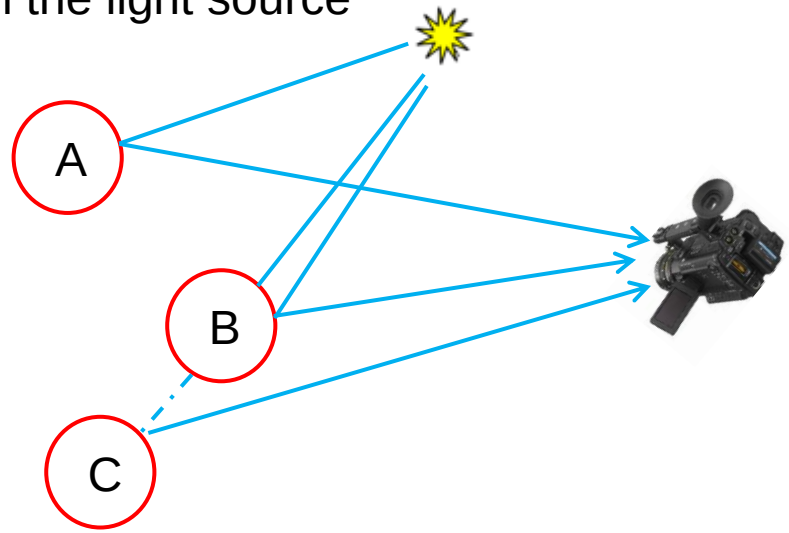


Shadow mapping

- Planar shadows can offer a nice effect, but are limited in several ways
  - Limited to planar receivers
  - No self-shadowing
  - Anti-shadows

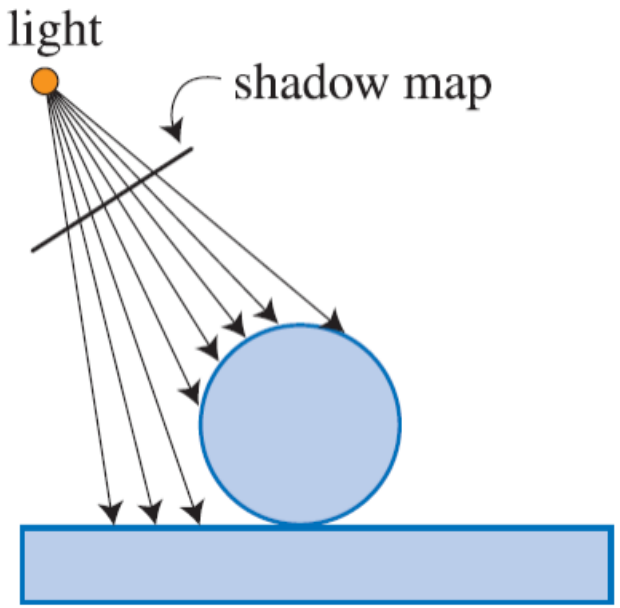


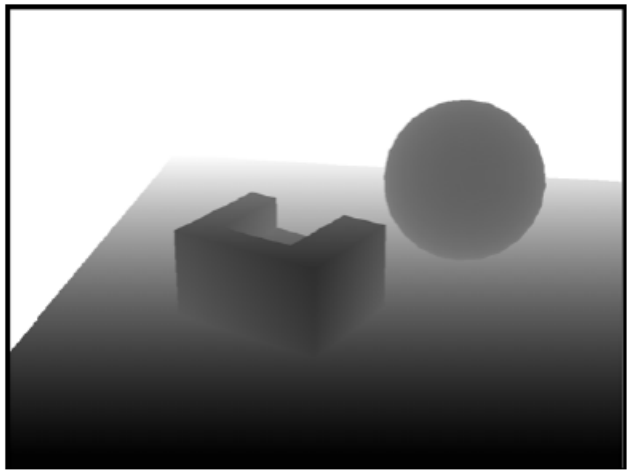
- In *shadow mapping*, one models the shadow casting problem as a *visibility problem*, but in this case, visibility is determined from the **light source**
- Shadow mapping classifies points follows:
  - Illuminated: visible from the light source
  - In shadow: not visible from the light source



Shadow mapping

- The method employs a *shadow map*, which is a depth rendering of the scene from the perspective of the **light** source.
- Each pixel in the shadow map stores a distance value from the scene point to the light source.



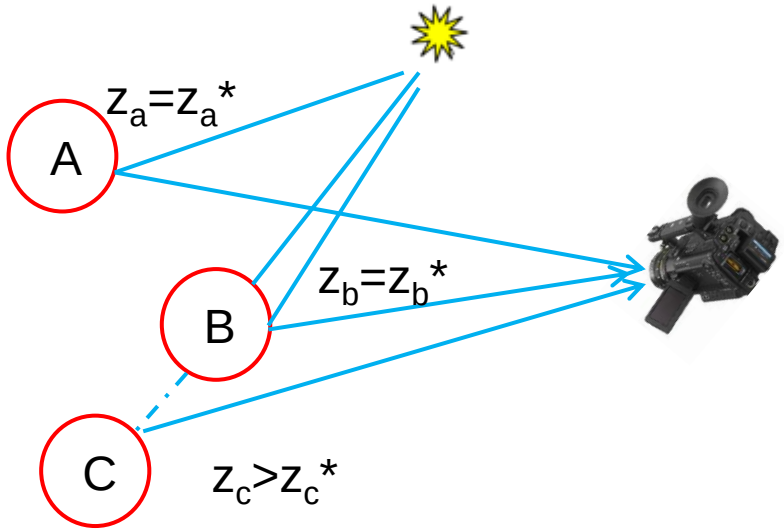


shadow map

[Akenine-Moeller et al.: Real-time Rendering]

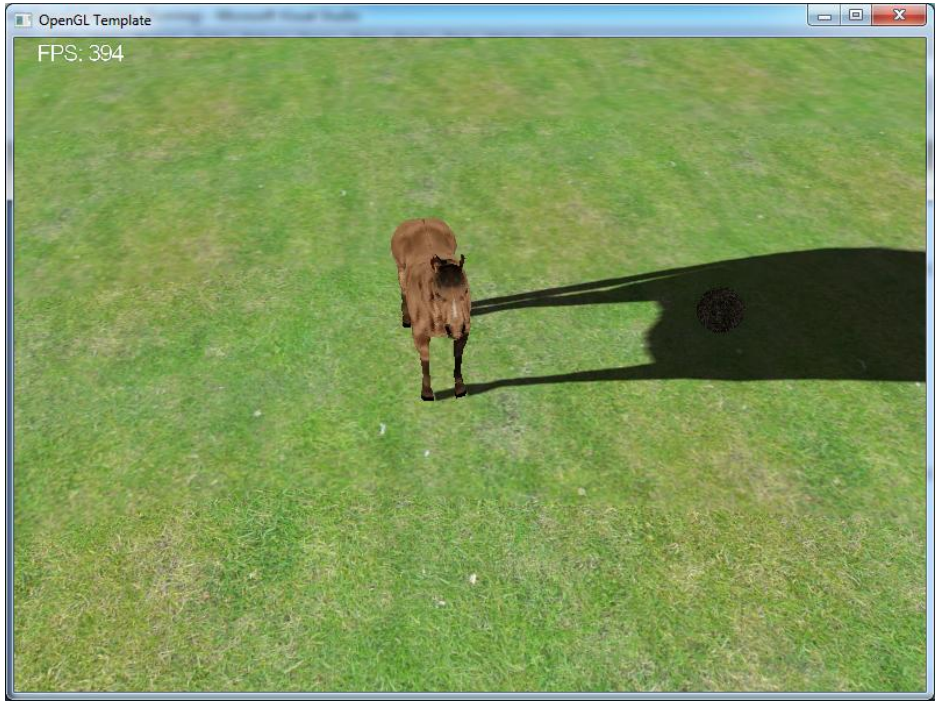
Shadow mapping

- Then, when the scene is rendered, each pixel's distance to the light is compared to the value in the shadow map.
- If the distances are equal, the fragment is illuminated
- However, if the distance is larger than the value in the shadow map, the fragment is in shadow
- For example, below, for a point on C visible to the camera, the distance to the light source is z_c. However, for this pixel, the shadow map has a smaller distance coming from surface B. Therefore we know the fragment is in shadow.



Shadow mapping

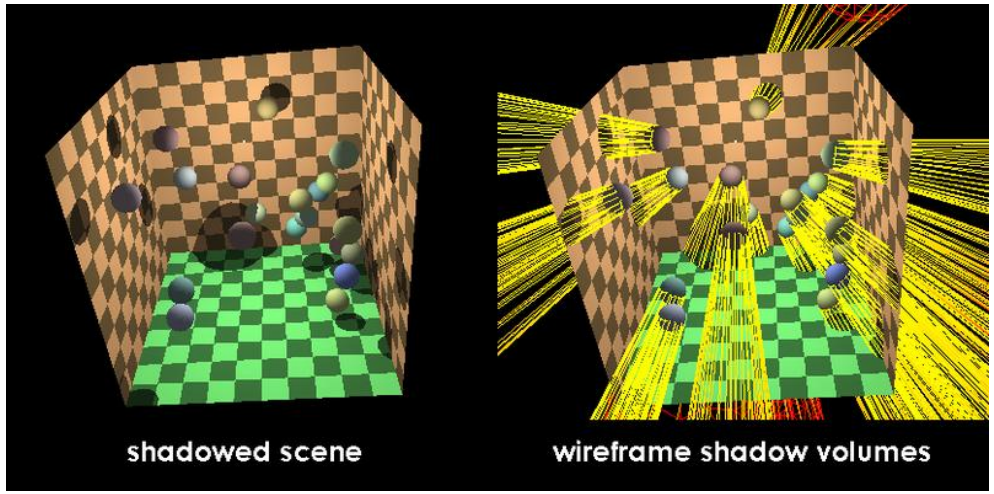
- Shadow mapping allows self-shadows and shadows on arbitrary shapes



- Shadow mapping works well but accuracy is limited to the resolution of the buffer used to generate the shadow map

Shadow volumes

- Shadow volumes construct a polyhedral representation of the region occluded from a light source.
- A shadow volume divides the scene in two regions:
  - Those in shadow
  - Those that are not in shadow
  - This is based on the boundary of the object as seen from the light source



Shadow volumes in Doom 3

More details at http://ogldev.atspace.co.uk/www/tutorial40/tutorial40.html

Instanced rendering

- Sometimes one would like to render *many* instances of the same object.
- Examples:
  - Blades of grass
  - Forest of trees
  - Army
- In the client program, one could loop over each object and render it:

```
for (int i = 0; i < N; i++)
    glDrawArrays(GL_TRIANGLES, 0, m_iNumVertices);
```

- However, each call to `glDrawArrays` carries some overhead. Instead, one can render the entire set with a single call

```
glDrawArraysInstanced(GL_TRIANGLES, 0, m_iNumVertices, N);
```

Instanced rendering

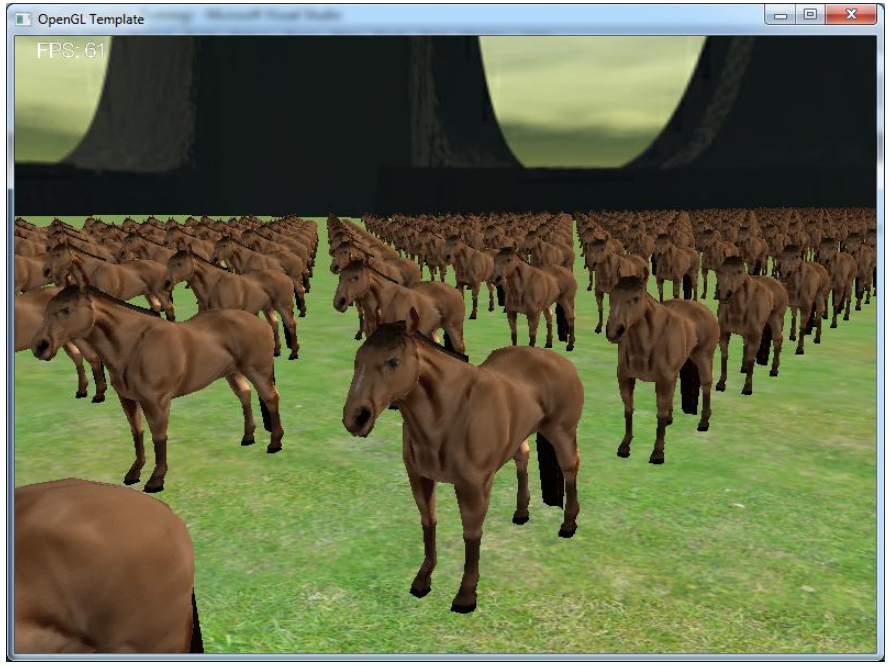
- In the vertex shader, there is a built-in variable called `gl_InstanceID`
- This variable allows the programmer to treat each instance differently, for example, by applying a transformation

vertex shader

```
float x = gl_InstanceID % 32;
float z = gl_InstanceID / 32;
vec3 shift = 10*vec3(x-16.0, 0, z-16.0);
gl_Position = matrices.projMatrix * matrices.modelViewMatrix *
              vec4(inPosition + shift, 1.0f);
```

Instanced rendering

- Example: A herd of 1000 horses!
- Each horse: 1581 triangles



- Instanced rendering: 60 FPS
- Non-instanced: 40 FPS

Aliasing and jaggies

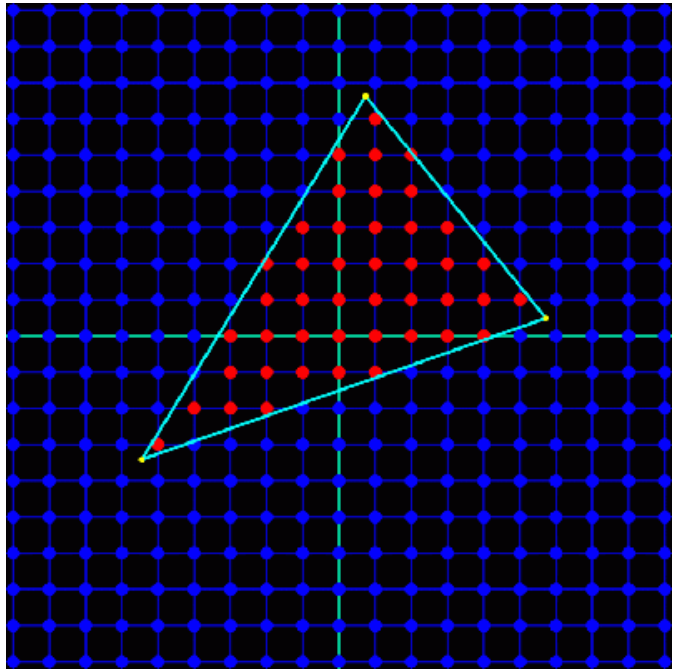
- We've seen before that aliasing can be detrimental during rendering.
- Mipmapping and anisotropic filtering help considerably in rendering high quality texture mapping with minimal aliasing. However, aliasing can still be visible on renderings, particularly along primitive edges.

Zoomed in 200%

Jaggies – staircasing artefact resulting from limited framebuffer resolution

Aliasing

- As we said before, aliasing results from undersampling a signal.
- In the case below, the signal is a triangle with well defined edges.
- However, the set of fragments on the screen do not have enough resolution to capture the fine detail of the edges. This results in jaggies.



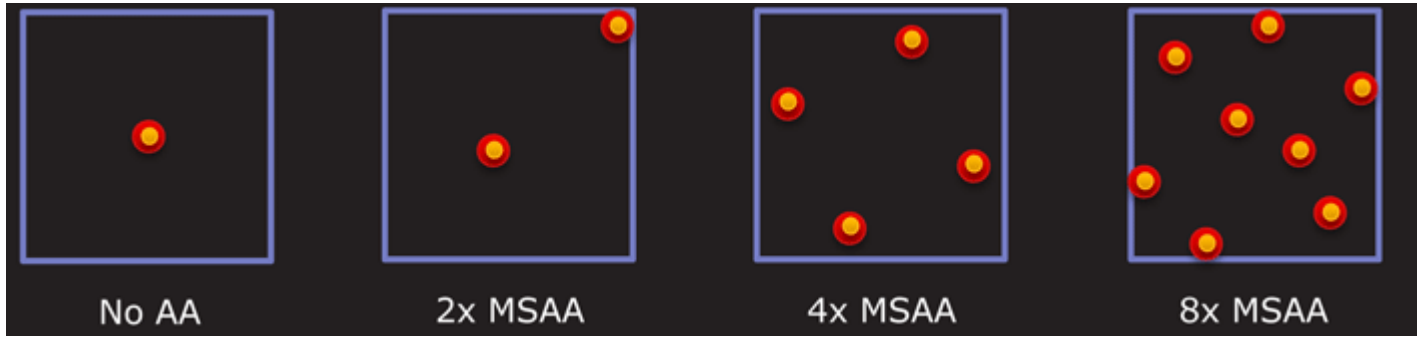
http://mynameismjp.files.wordpress.com/2012/10/coverage.png

Supersample anti-aliasing (SSAA)

- One way to improve image quality is to simply render to a higher resolution framebuffer, and then downsample the image to the window size.
- Downsampling involves combining nearby pixels, which makes jaggies look much smoother.
- Arguably, this approach produces the best anti-aliasing. However, it is costly, computationally. For example, rendering to a framebuffer that is a factor of two in width and height larger will require 4x fragments to be rendered in the fragment shader.
- This is often unworkable for real-time rendering.

Multi-sample anti-aliasing (MSAA)

- Instead, one can render multiple samples around a fragment being rendered. The number of samples is configurable, as well as their locations.
- Multi-sampling anti-aliasing (MSAA) generates several fragments with slightly different sub-pixel locations, and averages the result.
- MSAA runs the fragment program *just once* per fragment rasterised.
- The locations are pseudo-randomly positioned throughout the pixel region.



http://www.anandtech.com/show/4061/amds-radeon-hd-6970-radeon-hd-6950/10

Turning MSAA on

- When creating the window, allocate an MSAA buffer and then enable GL_MULTISAMPLE

```
const int iPixelFormatAttribList[] =
{
    WGL_DRAW_TO_WINDOW_ARB, GL_TRUE,
    WGL_SUPPORT_OPENGL_ARB, GL_TRUE,
    WGL_DOUBLE_BUFFER_ARB, GL_TRUE,
    WGL_ACCELERATION_ARB, WGL_FULL_ACCELERATION_ARB
    WGL_PIXEL_TYPE_ARB, WGL_TYPE_RGBA_ARB,
    WGL_COLOR_BITS_ARB, 32,
    WGL_DEPTH_BITS_ARB, 24,
    WGL_STENCIL_BITS_ARB, 8,
    WGL_SAMPLE_BUFFERS_ARB, 1, //Number of buffers
    WGL_SAMPLES_ARB, 8,        //Number of samples
    0 // End of attributes list
};
```

WGL_SAMPLE_BUFFERS_ARB is always 1
WGL_SAMPLES_ARB defines the number of multi-samples to use

After the rendering context is created, call
`glEnable( GL_MULTISAMPLE );`

No multi-sampling vs 8x MSAA

- Zoomed in 200%

No multisampling, 875 FPS

8x MSAA, 300 FPS

Module Evaluation Survey

- A formal way to gather your feedback to improve the student experience
- More information on the Student Hub
- Access all open surveys on the Student Survey Portal or via the QR code (you will also be emailed a link)

Demo Coursework

- Demo coursework submission area will be open imminently
- Due Sunday 5:00 pm, 12th April
- Suggestion: upload *well before* the last minute! Server may get busy near the deadline. “The server was busy” or “My internet connection was slow” are not a valid excuses for a late submission.

- **Maximum file size is 200 MB** (.zip file)
  - Delete the .vs directory
  - ⇨ This is often huge and can automatically be regenerated by Visual Studio
  - ⇨ Build->Clean Solution before compression will also reduce the size
  - ⇨ Hopefully the interim submission got you used to this process

- Marks for interim coursework will be released in the next two weeks

What’s in the package

- Three directories:
  - Bin
  - Documentation
  - Source

Name	Date modified	Type	Size
Bin	23/03/2014 17:09	File folder	
Documentation	23/03/2014 17:09	File folder	
Source	23/03/2014 17:09	File folder	

Source directory

- All source code and related files. Make sure the project compiles with a simple ‘F5’ in Visual Studio and the program runs properly.

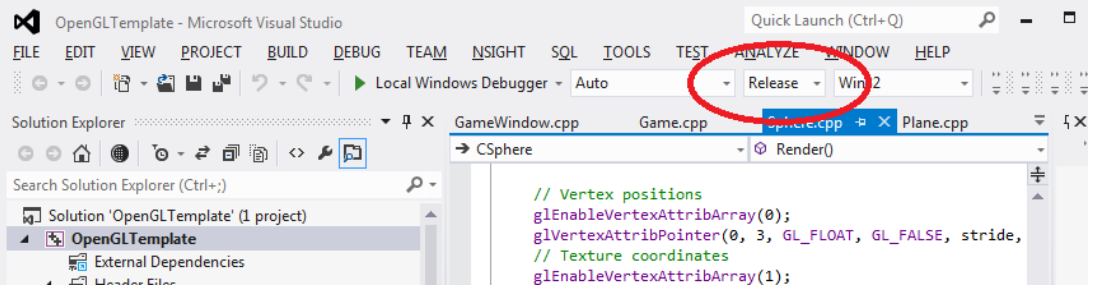
Name	Date modified	Type	Size
Debug	27/02/2017 12:06	File folder	
OpenGLTemplate	13/03/2017 10:33	File folder	
Release	29/01/2017 13:19	File folder	
OpenGLTemplate.sln	17/01/2017 11:32	Microsoft Visual S...	1 KB

Bin directory

- The executable and any DLLs or resources required to run. Test by doing a **double click** on the .exe file. Will require duplicating the resources directory.

resources	30/11/2016 12:25	File folder	
Assimp32.dll	25/10/2012 21:34	Application extens...	6,909 KB
fmod.dll	20/01/2017 17:50	Application extens...	1,433 KB
fmod64.dll	20/01/2017 17:50	Application extens...	1,653 KB
fmodL.dll	20/01/2017 17:50	Application extens...	1,570 KB
fmodL64.dll	20/01/2017 17:50	Application extens...	1,808 KB
FreeImage.dll	31/01/2013 11:02	Application extens...	2,419 KB
freetype2410.dll	31/01/2013 23:11	Application extens...	399 KB
glew32.dll	22/07/2013 15:12	Application extens...	330 KB
glew32mx.dll	22/07/2013 15:12	Application extens...	225 KB
OpenGLTemplate.exe	13/03/2017 10:32	Application	77 KB

⇨ Compile the program in Release mode for your this version, and copy all files from x64\Release after compilation. Then duplicate the resources.



- If you developed your coursework on your laptop or machine at home, it may be a good idea to test your executable on a different machine – like one of the lab machines (that have Visual Studio installed).

Documentation directory

Empty for now – will contain your report

Marking

- The demo coursework marking sheet is also on Moodle – use this as a checklist to make sure everything has been implemented. It is based precisely on the coursework instructions.
- Increased complexity will score higher.
- The coursework has been designed to tie the entire module together – the vast majority of the content has been discussed in lectures. It provides an opportunity for you to demonstrate an understanding of the material in development of a working computer game!
- And hopefully you have a demo you can show as part of your portfolio of work done here at City

A few final tips

- Check to make sure your far clipping plane isn't clipping the scene. If so, increase it.
- Demonstrate that you know how to apply texture mapping and lighting to the objects you create – this includes reasonable texture coordinates and normals during rendering.
- Make sure your ambient terms aren't too high (see slide 3 in this lecture)
- Show that you can program a shader
- If your camera is flipping around unnaturally, either
  - Change your path geometry so the camera doesn't flip
  - Implement a technique to control the camera up vector
- If your path is self-intersecting, change the centreline for less curvature
- Try to deliver something against each requirement rather than overdelivering on one requirement but skipping others
- Save some time to write a clear report

Demo

- In the last week's lecture and lab session, you will have an opportunity to show the class your graphics coursework. Attendance is *mandatory*.
- All presentations on Tuesday 14th April
- Presentations should only last a few minutes – no more than five minutes to stay on schedule.
- We will proceed randomly (unless requested otherwise) – timetable to follow next week.
- If there are problems attending in person, get in touch soon
- If you prefer, you can submit a video (by Sunday 12th)
  - Joining by Zoom for questions is desirable
- Prizes awarded based on voting by YOU!!
  - Best Game
  - Best Graphics